Ex3a: Model Planning and Building - Classification

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

1. File upload

```
from google.colab import files

uploaded = files.upload()
```

Choose Files WA_Fn-Us…er-Churn.csv

WA_Fn-UseC_-Telco-Customer-Churn.csv(text/csv) - 977501 bytes, last modified: 10/7/2026 - 100% done

```
Saving WA_Fn-UseC_-Telco-Customer-Churn.csv to WA_Fn-UseC_-Telco-Customer-Churn.csv
```

```
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	

5 rows × 21 columns

```
print("Dataset Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Shape: (7043, 21)

Columns:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'Mult

Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
df['Churn'].value_counts()
```

Churn	count
No	5174
Yes	1869

dtype: int64

Separate features and target

```
X = df.drop('Churn', axis=1)
y = df['Churn']

print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (7043, 20)
Target shape: (7043,)
```

2. Split into Training and Testing data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (5634, 20)
Testing data: (1409, 20)
```

Model Building 3. Train a Logistic Regression classifier and predict the customer churn on the test dataset.

```
df = df.drop('customerID', axis=1)

df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df = df.dropna()
df = pd.get_dummies(df, drop_first=True)
X = df.drop('Churn_Yes', axis=1)
y = df['Churn_Yes']

print("X shape:", X.shape)
print("y shape:", y.shape)
print("\nData types:")
print(X.dtypes.value_counts())
```

```
X shape: (7032, 30)
y shape: (7032,)

Data types:
bool       26
int64       2
float64     2
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training:", X_train.shape)
print("Testing:", X_test.shape)
```

```
Training: (5625, 30)
Testing: (1407, 30)
```

```
log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

print("Logistic Regression predictions:")
print(y_pred_log[:10])
```

```
Logistic Regression predictions:
[False  True False False False False False False  True False]
/usr/local/lib/python3.13/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning:
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

4. Train a Decision Tree Classifier and predict the customer churn on the test dataset.

```
dt_model = DecisionTreeClassifier(
    random_state=42
)

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)

print("Decision Tree predictions:")
print(y_pred_dt[:10])
```

```
Decision Tree predictions:
[False  True False False False  True False False  True False]
```

5. Train a Random Forest Classifier and predict the customer churn on the test dataset.

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("Random Forest predictions:")
print(y_pred_rf[:10])
```

```
Random Forest predictions:
[False  True False False False False False False  True False]
```

Model Evaluation 6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, F1-Score

```
def evaluate_model(model_name, y_test, y_pred):
    cm = confusion_matrix(y_test, y_pred)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print("\n", model_name)
    print("-" * 40)
    print("Confusion Matrix:")
    print(cm)
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1-Score :", round(f1, 4))

    return accuracy, precision, recall, f1
```

```
log_metrics = evaluate_model(
    "Logistic Regression",
    y_test,
    y_pred_log
)

dt_metrics = evaluate_model(
    "Decision Tree",
    y_test,
    y_pred_dt
)
```

```
rf_metrics = evaluate_model(
    "Random Forest",
    y_test,
    y_pred_rf
)
```

```
 Logistic Regression
----------------------------------------
Confusion Matrix:
[[916 117]
 [160 214]]
Accuracy : 0.8031
Precision: 0.6465
Recall   : 0.5722
F1-Score : 0.6071

 Decision Tree
----------------------------------------
Confusion Matrix:
[[838 195]
 [201 173]]
Accuracy : 0.7186
Precision: 0.4701
Recall   : 0.4626
F1-Score : 0.4663

 Random Forest
----------------------------------------
Confusion Matrix:
[[917 116]
 [180 194]]
Accuracy : 0.7896
Precision: 0.6258
Recall   : 0.5187
F1-Score : 0.5673
```

7. Compare the performance of all three models using a single bar chart showing the evaluation metrics

```
results = pd.DataFrame(
    [log_metrics, dt_metrics, rf_metrics],
    columns=["Accuracy", "Precision", "Recall", "F1-Score"],
    index=[
        "Logistic Regression",
        "Decision Tree",
        "Random Forest"
    ]
)

results
```

	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.803127	0.646526	0.572193	0.607092
Decision Tree	0.718550	0.470109	0.462567	0.466307
Random Forest	0.789623	0.625806	0.518717	0.567251

Next steps: Generate code with `results` New interactive sheet

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

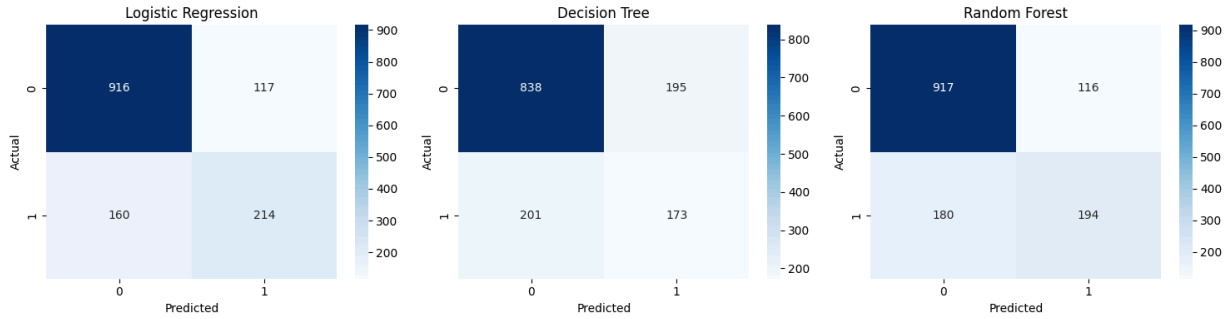
```
models = [
    ("Logistic Regression", y_pred_log),
    ("Decision Tree", y_pred_dt),
    ("Random Forest", y_pred_rf)
]

for ax, (name, y_pred) in zip(axes, models):
    cm = confusion_matrix(y_test, y_pred)

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        ax=ax
    )

    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

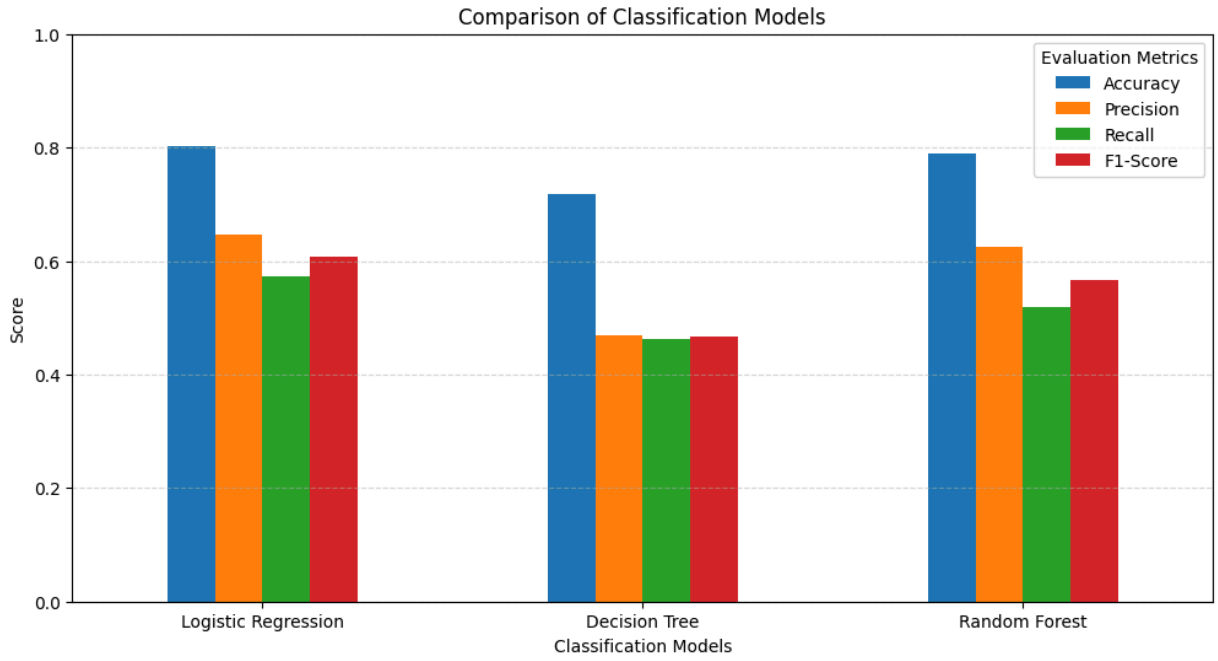
plt.tight_layout()
plt.show()
```




```
results.plot(
    kind='bar',
    figsize=(12, 6)
)

plt.title("Comparison of Classification Models")
plt.xlabel("Classification Models")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(title="Evaluation Metrics")
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```




```
best_model = results['F1-Score'].idxmax()
best_f1 = results['F1-Score'].max()

print("Best Performing Model:", best_model)
print("Best F1-Score:", round(best_f1, 4))
```

```
Best Performing Model: Logistic Regression
Best F1-Score: 0.6071
```

Double-click (or enter) to edit

8. Identify the best-performing model based on the evaluation results.

```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

feature_importance
```

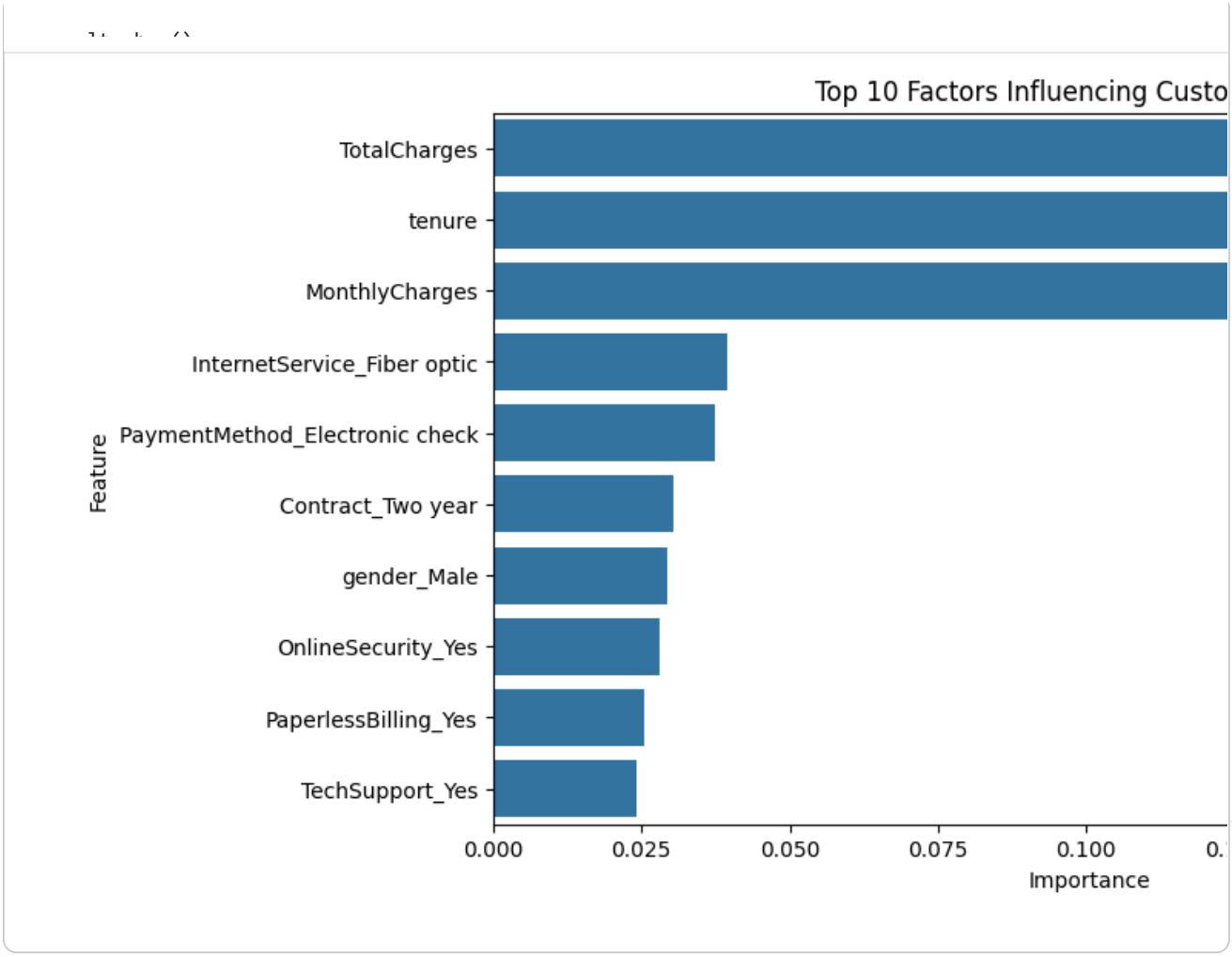
	Feature	Importance
3	TotalCharges	0.191435
1	tenure	0.171020
2	MonthlyCharges	0.168400
10	InternetService_Fiber optic	0.039481
28	PaymentMethod_Electronic check	0.037416
25	Contract_Two year	0.030529
4	gender_Male	0.029332
13	OnlineSecurity_Yes	0.028157
26	PaperlessBilling_Yes	0.025536
19	TechSupport_Yes	0.024114
5	Partner_Yes	0.023682
15	OnlineBackup_Yes	0.022932
24	Contract_One year	0.022836
0	SeniorCitizen	0.021035
6	Dependents_Yes	0.019846
9	MultipleLines_Yes	0.019588
17	DeviceProtection_Yes	0.018728
23	StreamingMovies_Yes	0.017707
21	StreamingTV_Yes	0.016523
27	PaymentMethod_Credit card (automatic)	0.013528
29	PaymentMethod_Mailed check	0.011774
16	DeviceProtection_No internet service	0.009089
12	OnlineSecurity_No internet service	0.007007
14	OnlineBackup_No internet service	0.005896
20	StreamingTV_No internet service	0.005404
18	TechSupport_No internet service	0.004460
7	PhoneService_Yes	0.004357
11	InternetService_No	0.003936
8	MultipleLines_No phone service	0.003872
22	StreamingMovies_No internet service	0.002380

Next steps: Generate code with `feature_importance` New interactive sheet

```
plt.figure(figsize=(10, 6))

sns.barplot(
    data=feature_importance.head(10),
    x='Importance',
    y='Feature'
)

plt.title("Top 10 Factors Influencing Customer Churn")
plt.xlabel("Importance")
plt.ylabel("Feature")
```



Business Recommendation

9. Based on the best-performing model, provide three recommendations that the telecommunications company can implement to reduce customer churn.

Recommendation 1 — Target high-risk customers Use the churn prediction model to identify customers with a high probability of churn and provide them with personalized retention offers before they discontinue